

Policy Gradient Methods

Policy Gradient Methods

- Policy Gradient methods bypass value functions as the primary decision-maker.
- Instead, they directly parameterize the policy and optimize it using gradient ascent.
- In Policy Gradient (PG) methods, we represent the policy π using a parameterized function (usually a neural network) with weights θ .
- The policy outputs a probability distribution over actions for a given state:

$$\pi_{\theta}(a \mid s) = \mathbb{P}[A_t = a \mid S_t = s, \theta]$$

Advantages over action-value methods like DQN:

1. DQN struggles with continuous actions because finding the maximum Q-value requires an exhaustive search or complex optimization at every single step.
 - PG methods simply output the parameters of a continuous distribution (like the mean and variance of a Gaussian).
2. Q-learning always learns a deterministic policy.
 - PG methods can learn optimal stochastic policies too.
3. Small changes in Q-values can cause the action selection to drastically flip, causing unstable training.
 - Small changes in policy parameters θ smoothly change the action probabilities.

The Policy Gradient Theorem (Derivation)

1. Defining Aim

- We want our agent to maximize its total reward.
- Let's call a full episode a *trajectory*, represented by τ .
- The total reward for a trajectory is $R(\tau)$.

- The probability of a specific trajectory happening, given our current policy weights θ , is $P(\tau \mid \theta)$.
- Our objective function $J(\theta)$ is simply the expected return over all possible trajectories:

$$J(\theta) = \int P(\tau \mid \theta) R(\tau) d\tau$$

2. Maximizing the Objective

- To maximize this objective using gradient ascent, we need to find the gradient (derivative) with respect to our policy parameters θ .
- Since the reward $R(\tau)$ comes from the environment and does not depend on θ , we only take the gradient of the probability:

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} P(\tau \mid \theta) R(\tau) d\tau$$

- Now we need this equation to be an expectation so we can estimate it by simply letting our agent play the game.
- We know the log-derivative trick:

$$\nabla \log(x) = \frac{\nabla x}{x}$$

- On rearranging, we get:

$$\nabla x = x \nabla \log(x)$$

- Now, replacing x with our probability $P(\tau \mid \theta)$:

$$\nabla_{\theta} P(\tau \mid \theta) = P(\tau \mid \theta) \nabla_{\theta} \log P(\tau \mid \theta)$$

- Substituting this back into our integral:

$$\nabla_{\theta} J(\theta) = \int P(\tau \mid \theta) \nabla_{\theta} \log P(\tau \mid \theta) R(\tau) d\tau$$

- Because we have $P(\tau \mid \theta)$ multiplying everything again, this is now an expectation:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau \mid \theta) R(\tau)]$$

3. What $\nabla_{\theta} \log P(\tau \mid \theta)$ actually is

- What is the probability of a trajectory $P(\tau \mid \theta)$? It is the product of three things:
 1. The probability of the starting state: $P(s_0)$
 2. The probability of the agent's actions (our policy): $\pi_{\theta}(a_t \mid s_t)$
 3. The probability of the environment's transitions: $P(s_{t+1} \mid s_t, a_t)$
- Therefore, on simplifying with logarithms and substituting:

$$\log P(\tau \mid \theta) = \log P(s_0) + \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t) + \sum_{t=0}^T \log P(s_{t+1} \mid s_t, a_t)$$

4. Eliminating the Environment Dynamics

- We take the gradient of the entire log sum with respect to θ .
- The environment's starting state (s_0) and transition probabilities ($P(s_{t+1} \mid s_t, a_t)$) have absolutely nothing to do with our policy weights θ .
- Therefore, their derivatives are exactly zero!
- All that survives the gradient is our policy:

$$\nabla_{\theta} \log P(\tau \mid \theta) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t)$$

5. The Final Equation

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) R(\tau) \right]$$

- To improve your policy, you take the gradient of the log probability of your actions and multiply it by the total reward $R(\tau)$.
- If the reward is high, the gradient steps in a direction that makes those actions more likely.
- If the reward is low or negative, it pushes those actions away.

The REINFORCE Algorithm

- REINFORCE is the most fundamental Monte Carlo Policy Gradient algorithm.
- It takes the Policy Gradient Theorem and applies it in a practical, sample-based way.
- Instead of computing the true mathematical expectation, the agent plays an entire episode to generate a sample trajectory.
- At each time step t , it updates the policy weights θ using the actual return from that time step onward (G_t).
- G_t acts as a scalar multiplier:
 - If the return G_t was highly positive, we take a big step in that direction, making that action much more likely in the future.
 - If the return G_t was negative (or very small), we push the probability of that action down.

The REINFORCE Update Rule:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) G_t$$

REINFORCE with Baseline and Variance Reduction

- The standard REINFORCE algorithm works, but it suffers from extreme variance.
- Because G_t depends on a long sequence of random actions and state transitions, the returns can fluctuate wildly from one episode to the next.
- We can significantly reduce this variance by subtracting a **baseline** $b(s)$ from the return.
- The baseline can be any function, as long as it does not depend on the action a .

The Baseline Update Rule:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - b(S_t))$$

Why the Baseline is Unbiased?

- Subtracting a random number from our return would not break the math and bias our gradient because the expected value of the baseline term is exactly zero.

Variance Reduction

- While any baseline works mathematically, the most effective baseline is the state-value function $V(s)$, which represents the average expected return from state s .
- When we substitute $V(s)$ into our equation, the multiplier becomes:

$$A_t = G_t - V(S_t)$$

This term is called the **Advantage**.

- If $G_t > V(S_t)$, the action we took resulted in a return better than average. The Advantage is positive, and the probability of that action is pushed heavily upwards.
- If $G_t < V(S_t)$, the action resulted in a return worse than average. The Advantage is negative, and the probability of that action is penalized and pushed down.
- By centering the returns around zero, the variance of the gradients shrinks dramatically, allowing the policy network to learn much faster and far more stably.